

1. QRDQN en Stable Baselines3 para el TFG

1.1. Resumen ejecutivo

La implementación de **QR-DQN** en el ecosistema de Stable Baselines vive en **SB3-Contrib**, no en el núcleo de Stable Baselines3, y convierte a DQN en un método **distribucional**: en vez de aprender un único escalar $Q(s, a)$, aprende una **distribución aproximada de retornos** por acción usando N cuantiles. En la práctica de SB3, esa distribución se representa como un tensor de forma $(\text{batch}, N, |\mathcal{A}|)$, se entrena con **quantile Huber loss**, y la acción codiciosa se elige por la **media de los cuantiles**, es decir, por el retorno esperado implícito en la distribución. Por tanto, QRDQN aprende más información que DQN, pero **no es risk-sensitive por defecto**: sigue actuando según la esperanza matemática salvo que se modifique explícitamente el criterio de decisión.

Desde la perspectiva teórica, QR-DQN nace para cerrar la brecha entre la teoría distribucional de Marc G. Bellemare, Will Dabney, y Rémi Munos en C51 y una implementación entrenable por descenso de gradiente. C51 fija los soportes z_i y aprende probabilidades; QR-DQN “transpose” esa idea: fija probabilidades uniformes y aprende las **localizaciones** de los cuantiles. Esto evita la proyección categórica de C51 y elimina la necesidad de fijar manualmente cotas como $V_{\min}$ y $V_{\max}$.

En el repositorio `yeaight7/TFG-CYBER_AI`, QRDQN es el algoritmo principal para un problema discreto binario de ciberdefensa sobre **CICIDS2017**, con observaciones de **152 dimensiones** y acciones **0=PERMIT**, **1=BLOCK**. La arquitectura elegida es `MlpPolicy` con `net_arch=[512, 256]`, `learning_rate=1e-4`, `gamma=0.99`, `tau=1.0`, `train_freq` de 50 o 100 y `gradient_steps` de 10 o 20 según preset. El mejor artefacto consolidado del repositorio es el run `C03_qrdqn_cicids2017_canonical_full_random_20260223_232439`, con accuracy 0.99859 y F1 de ataque 0.99876 en random split; sin embargo, la validación dura por día/CSV cae de forma importante, lo que demuestra que la generalización real es bastante más difícil que el escenario aleatorio.

La conclusión práctica para el TFG es clara: QRDQN está bien alineado con el problema porque el espacio de acciones es discreto y porque el aprendizaje distribucional puede capturar mejor colas, multimodalidad y asimetrías del retorno inducidas por una recompensa con falsos negativos muy penalizados. Pero, en SB3, esa ventaja depende mucho de detalles de implementación: **número de cuantiles**, rapidez del decaimiento de ε , tamaño del batch, frecuencia de actualización de la red objetivo y diseño de la recompensa.

1.2. Supuestos y lagunas del repositorio

- La versión exacta de SB3 y SB3-Contrib no está fijada, solo un mínimo: `stable-baselines3>=2.3` y `sb3-contrib>=2.3`. En SB3-Contrib v2.3.0 cambió el default de `learning_starts` de 50 000 a 100.
- `n_quantiles` no se fija explícitamente en el script de entrenamiento; se infiere que se usa el default oficial: **200 cuantiles**.
- Los parámetros de exploración tampoco se fijan explícitamente. Se heredan los defaults de QRDQN en SB3-Contrib: `exploration_fraction=0.005`, `exploration_initial_eps=1.0` y `exploration_final_eps=0.01`. Con 100 000 timesteps, la caída lineal de ε termina aproximadamente en los **primeros 500 pasos**; con 25 000 timesteps, en los **primeros 125 pasos**.

- El script `train_rl_defender.py` contiene una inconsistencia documental interna: en la ayuda de `-timesteps` se habla de un default de “500k full, 5k fast”, pero el código efectivo en `main()` asigna 25_000 para `fast` y 100_000 para `full`.
- Los resultados históricos no coinciden necesariamente con los defaults actuales del código. El reward actual por defecto usa `fp=-1.5`, mientras que el mejor run histórico consolidado usó `fp=-2.0`.
- La validación `leave-one-exact-CSV-out` está implementada, pero no hay un artefacto agregado comprometido en `runs/validation/`.

Como hallazgo sintético del repo: el pipeline actual trabaja con CICIDS2017, un esquema canónico de **76 features**, una máscara de missingness de otras **76**, observación total de **152 dimensiones**, y un entorno Gymnasium con acción discreta binaria.

1.3. Fundamentos teóricos y matemáticos

La motivación de **distributional RL** es que el retorno futuro desde un estado-acción no es un número fijo, sino una **variable aleatoria**. DQN clásico aprende solo su expectativa $Q(s, a) = \mathbb{E}[Z(s, a)]$. El enfoque distribucional intenta aproximar la ley completa de $Z(s, a)$, lo que permite representar colas, multimodalidad y asimetría.

En C51, la distribución se aproxima con una combinación discreta de **soportes fijos** $z_1, \dots, z_N$ y **probabilidades aprendidas** q_i . QR-DQN invierte esa parametrización: fija probabilidades uniformes $1/N$ y aprende las **localizaciones** $\theta_i(s, a)$. Formalmente:

$$Z_\theta(s, a) = \frac{1}{N} \sum_{i=1}^N \delta_{\theta_i(s, a)},$$

donde cada θ_i representa una localización cuantil y δ es una masa de Dirac.

La **pinball loss** para un único cuantil τ :

$$\rho_\tau(u) = u \left(\tau - \mathbf{1}_{\{u < 0\}} \right).$$

Como la pinball loss no es suave en cero, el paper propone la **quantile Huber loss**:

$$L_\kappa(u) = \begin{cases} \frac{1}{2}u^2, & |u| \leq \kappa \\ \kappa \left(|u| - \frac{1}{2}\kappa \right), & |u| > \kappa \end{cases} \quad \rho_\tau^\kappa(u) = |\tau - \mathbf{1}_{\{u < 0\}}| L_\kappa(u).$$

El **Bellman update** distribucional en control:

$$\mathcal{T}Z(s, a) \stackrel{D}{=} r + \gamma Z(s', a^*), \quad a^* = \arg \max_{a'} \mathbb{E}[Z(s', a')].$$

1.4. Lógica algorítmica e implementación en SB3

A nivel algorítmico, la lógica de QRDQN en SB3 es la de un **off-policy method** clásico: recolecta transiciones en un replay buffer, espera hasta `learning_starts`, entrena cada `train_freq` pasos durante `gradient_steps` iteraciones y actualiza la red objetivo cada `target_update_interval`.

Cuadro 1: Comparación entre DQN, C51 y QRDQN.

Atributo	DQN	C51	QRDQN
Qué modela	Esperanza $Q(s, a)$	Distribución cate- górica	Distribución por cuantiles
Representación	1 escalar por acción	Soportes fijos + probs aprendidas	Probs uniformes + locali- zaciones aprendidas
Salida por acción	1 valor	N probabilidades	N cuantiles
Necesita $V_{\min}, V_{\max}$	No	Sí	No
Pérdida	TD/Huber escalar	KL a proyección cate- górica	Quantile Huber
Política greedy	$\arg \max Q$	$\arg \max \mathbb{E}[Z]$	$\arg \max \mathbb{E}[Z]$
Ventaja principal	Simplicidad	Distribucional con coste moderado	Distribucional más flexi- ble, sin soportes fijos
Inconveniente princi- pal	Pierde información de dispersión	Requiere proyec- ción y cotas	Cabeza de salida más grande y más coste

```

obs_t
-> selección de acción
-> env.step
-> guardar transición en replay buffer
-> [num_timesteps > learning_starts?]
    no -> obs_t
    sí -> [se alcanzó train_freq?]
        no -> obs_t
        sí -> muestra minibatch
            -> Z_next = target_net(next_obs)
            -> a* = argmax_a media_tau Z_next
            -> Z_target = r + gamma * (1-done) * Z_next[a*]
            -> Z_current = online_net(obs, action)
            -> quantile_huber_loss
            -> Adam / backprop
            -> [target_update_interval?]
                sí -> polyak_update -> obs_t
                no -> obs_t

```

El detalle más importante de la implementación oficial es **cómo se elige la acción bootstrap**. SB3 calcula los cuantiles del siguiente estado con la red objetivo, toma la **media** sobre el eje de cuantiles, aplica **argmax** sobre acciones y luego recoge los cuantiles de esa acción ganadora para formar el target.

Listing 1: Pseudocódigo del entrenamiento QRDQN en SB3-Contrib

```

# Pseudocódigo fiel al entrenamiento de SB3-Contrib QRDQN
Z_next = target_net(next_obs) # (B, N, A)
a_star = argmax_a(mean_tau(Z_next)) # acción greedy por
    esperanza
Z_boot = select_action_quantiles(Z_next, a_star) # (B, N)
Z_target = r + (1 - done) * gamma * Z_boot

Z_current = select_action_quantiles(online_net(obs), action)
loss = quantile_huber_loss(Z_current, Z_target)

```

En cuanto al **replay buffer**, QRDQN hereda la lógica genérica de `OffPolicyAlgorithm`: **1-step targets** y replay uniforme, **sin prioritized replay nativo**.

La **red objetivo** se actualiza con `polyak_update`. Con `tau=1.0` (valor usado en el repo) esto equivale a una **copia dura** cada `target_update_interval`.

La **exploración** es ε -greedy con programación lineal entre `exploration_initial_eps` y `exploration_final_eps` a lo largo de `exploration_fraction`.

El **optimizador** por defecto es Adam. QRDQN inyecta `optimizer_kwargs={.eps": 0.01 / batch_size}`. Con `batch_size=2048`, el ε implícito del Adam es $\approx 4.88 \times 10^{-6}$; con `batch_size=512`, $\approx 1.95 \times 10^{-5}$.

1.5. Arquitectura de red y parámetros de la API

La política oficial `QRDQNPoly` construye dos redes: `quantile_net` y `quantile_net_target`. Para observaciones vectoriales, el extractor por defecto es `FlattenExtractor` y la arquitectura MLP por defecto es `[64, 64]`. La capa final genera `action_dim * n_quantiles` salidas reordenadas a forma `(-1, n_quantiles, n_actions)`.

Observacion x -> FlattenExtractor -> MLP hidden 1 -> MLP hidden 2
-> Linear(A*N) -> reshape(batch, N, A) -> media sobre N -> argmax accion

En el repo, la arquitectura concreta es `MlpPolicy` con `net_arch=[512, 256]`. Con `n_quantiles=200` (default), el cabezal de salida tiene **400 neuronas** (2 acciones $\times$ 200 cuantiles). La forma de salida es `(B, 200, 2)`.

El recuento de parámetros de la red online para una MLP vectorial es:

$$(dh_1 + h_1) + (h_1h_2 + h_2) + (h_2AN + AN),$$

donde d es la dimensión de observación, A el número de acciones y N el número de cuantiles. Con $d = 152$, $A = 2$, $N = 200$, $h_1 = 512$, $h_2 = 256$, el total es **312 464 parámetros** en la red online.

1.5.1. Parámetros del constructor QRDQN

1.5.2. Parámetros internos de QRDQNPoly

1.5.3. Fragmento relevante del repositorio

```
model = QRDQN(
    "MlpPolicy",
    vec_env,
    seed=seed,
    policy_kwargs=dict(net_arch=[512, 256]),
    learning_rate=lr,
    buffer_size=min(200_000, max(total_timesteps, 10_000)),
    batch_size=batch_size,
    gradient_steps=gradient_steps,
    gamma=0.99,
    tau=1.0,
    train_freq=train_freq,
    target_update_interval=target_update_interval,
    verbose=1,
    device=device,
    tensorboard_log=tb_log_dir,
)
```

Cuadro 2: Parámetros del constructor QRDQN y su estado en el repositorio.

Parámetro	Default	Papel	Valores típicos / efecto	En el repo
policy	—	tipo de política	depende de observación	Sí: "MlpPolicy"
env	—	entorno Gym/Gymnasium o VecEnv	obligatorio	Sí: DummyVecEnv(Monitor(...))
learning_rate	5e-5	ritmo de aprendizaje	más alto acelera, pero arriesga inestabilidad	Sí: 1e-4
buffer_size	1_000_000	tamaño máximo del re-play buffer	buffers grandes estabilizan, consumen RAM	Sí: cap en 200k
learning_starts	100	warm-up antes de entrenar	demasiado bajo: targets ruidosos	Implicito: default 100
batch_size	32	tamaño de minibatch	mayor batch reduce varianza	Sí: 512 / 2048
tau	1.0	soft update de target; 1.0 = hard copy	<1 blando; 1 duro	Sí: 1.0
gamma	0.99	descuento	más alto: horizonte largo	Sí: 0.99
train_freq	4	frecuencia de updates	controla cadencia entre datos y gradientes	Sí: 50 / 100
gradient_steps	1	nº de pasos de gradiente por ciclo	más alto = más compute	Sí: 10 / 20
replay_buffer_class	None	clase de buffer	custom buffer si se quiere extender	No
optimize_memory_usage	False	variante memory-efficient	ahorra memoria, añade complejidad	No
target_update_interval	10000	cada cuántos pasos se actualiza target	pequeño = más reactivo; grande = más estable	Sí: 1000 / 10000
exploration_fraction	0.005	fracción donde cae ϵ	más grande = exploración prolongada	Implicito
exploration_initial_eps	0.0	ϵ inicial	normalmente alto	Implicito
exploration_final_eps	0.01	ϵ final	más alto = más exploración residual	Implicito
policy_kwargs	None	kwargs internos de la política	donde se pasa <code>net_arch</code> , <code>n_quantiles</code>	Sí
seed	None	semilla global	clave para reproducibilidad local	Sí: 42 por defecto CLI
device	."auto"	CPU / CUDA	afecta velocidad y reproducibilidad	Sí: CPU o CUDA detectado

Cuadro 3: Parámetros de QRDQNPolicy propagados vía policy_kwargs.

Parámetro	Default oficial	Papel	En el repo
n_quantiles	200	nº de cuantiles por acción; afecta resolución y coste	Implicito
net_arch	[64,64] en MLP	profundidad/anchura MLP	Sí: [512,256]
activation_fn	ReLU	no linealidad	Implicito
features_extractor_class	FlattenExtractor	extractor para observación vectorial	Implicito
normalize_images	True	relevante en observaciones de imagen	Implicito, irrelevante aquí
optimizer_class	Adam	optimizador	Implicito
optimizer_kwargs	QRDQN eps=0.01/batch_size	detalle fino de estabilidad	Implicito

1.6. Consideraciones prácticas, evaluación y reproducibilidad

QRDQN suele ser una buena elección frente a DQN cuando el problema tiene **acciones discretas** y se sospecha que el retorno no está bien descrito por una media. Frente a C51, QRDQN resulta especialmente cómodo cuando no se quieren elegir manualmente cotas de soporte.

En SB3, sin embargo, esa superioridad no es automática. La guía oficial recomienda hacer tuning cuantitativo, evaluar en un entorno separado y validar muy bien terminaciones y wrappers.

El gran resultado positivo del repo es el rendimiento en random split: C03_qrdqn_cicids2017_canon reporta accuracy 0.99859 y F1 de ataque 0.99876. El hallazgo científicamente más valioso es negativo: en la validación dura por día/CSV (Check C) la accuracy baja a 0.84135 y el recall de ataque a 0.52954. Esto permite defender algo muy sólido: **el modelo aprende muy bien dentro del régimen i.i.d. aproximado del split aleatorio, pero generalizar entre días y capturas es mucho más difícil.**

Los **dos hiperparámetros críticos** a revisar en primer lugar son:

- `exploration_fraction`: al heredarse el default 0.005 puede que se explore demasiado poco durante casi todo el entrenamiento.
- `n_quantiles`: 200 es un default razonable en Atari pero quizá no sea óptimo para un problema tabular/vectorial con dos acciones.

Para visualizar la salida distribucional en una defensa:

Accion PERMIT:

q05 = -3.2 q50 = 0.1 q95 = 1.7 media $\approx$ -0.2

Accion BLOCK:

q05 = -0.9 q50 = 0.8 q95 = 1.1 media $\approx$ 0.7

Decision greedy de SB3: BLOCK

Interpretacion: BLOCK tiene mejor retorno esperado y menor dispersión.

Utilidad mínima para extraer curvas de cuantiles programáticamente:

```
import torch
import numpy as np

def get_action_quantiles(model, obs_np):
    obs_t = torch.as_tensor(obs_np[None], device=model.device, dtype=
        torch.float32)
    with torch.no_grad():
        z = model.quantile_net(obs_t)      # (1, N, A)
    z = z.squeeze(0).cpu().numpy()        # (N, A)
    q_mean = z.mean(axis=0)               # (A,)
    return z, q_mean
```

1.7. Preguntas de tribunal y respuestas breves

¿Qué aprende QRDQN que DQN no aprende? DQN aprende solo el retorno esperado por acción. QRDQN aprende una aproximación de la **distribución completa de retornos** mediante N cuantiles por acción.

¿En qué se diferencia QRDQN de C51 en una frase? C51 fija los soportes y aprende probabilidades; QRDQN fija probabilidades uniformes y aprende las localizaciones de los cuantiles. Por eso QRDQN no necesita $V_{\min}$ ni $V_{\max}$.

Si QRDQN aprende distribución, ¿por qué no es “risk-sensitive”? Porque en SB3 la acción se elige con `argmax` de la **media** de cuantiles. La distribución se aprende y se puede analizar, pero la política por defecto sigue maximizando esperanza.

¿Tu implementación usa prioritized replay, Double DQN o dueling? No de forma nativa. El QRDQN de SB3-Contrib usa `ReplayBuffer` estándar y muestreo uniforme.

¿Cuál es el hiperparámetro más “silencioso” de tu implementación? Probablemente `exploration_fraction`, porque no se fija en el repo y el default 0.005 implica que ϵ cae muy deprisa.

¿Por qué no basta con reportar accuracy de random split? Porque el propio repositorio muestra que el rendimiento en split aleatorio es casi perfecto, pero cae de forma notable en el split duro por día/CSV. Eso indica que el verdadero reto es la **generalización fuera de distribución**.

¿Cuál sería una mejora inmediata para la memoria del TFG? Fijar en texto y código los parámetros implícitos heredados de SB3-Contrib: versión exacta, `n_quantiles`, exploración y `optimizer_kwargs`.